```
        secureRandom.nextBytes(bytes);    #A
        return Base64url.encode(bytes);    #A
    }

    @Override
    public String create(Request request, Token token) {
        var tokenId = randomId();    #A
        var attrs = new
JSONObject(token.attributes).toString();    #B

        database.updateUnique("INSERT INTO " +
            "tokens(token_id, user_id, expiry, attributes) "
+
            "VALUES(?, ?, ?, ?)", tokenId, token.username,
                token.expiry, attrs);

        return tokenId;
    }

    @Override
    public Optional<Token> read(Request request, String
tokenId) {
        return database.findOptional(this::readToken,    #C
                "SELECT user_id, expiry, attributes " +
                "FROM tokens WHERE token_id = ?", tokenId);
    }

    private Token readToken(ResultSet resultSet)     #C
            throws SQLException {    #C
        var username = resultSet.getString(1);    #C
        var expiry =
resultSet.getTimestamp(2).toInstant();     #C
        var json = new JSONObject(resultSet.getString(3));
#C

        var token = new Token(expiry, username);    #C
        for (var key : json.keySet()) {    #C
            token.attributes.put(key,
json.getString(key));    #C
```